LOGISTIC REGRESSION PROJECT REPORT

Diabetes Prediction using Kaggle Data

Project	Diabetes Prediction using Kaggle Data
Student	Pallavi
Date	30 October 2026
Dataset	Pima Indians Diabetes Database (Kaggle), file: diabetes.csv
Tools	Python, pandas, NumPy, scikit-learn, Matplotlib, Seaborn

Objective	Dataset	Model
Predict Outcome: 0 = No diabetes, 1 = Diabetes.	Pima Indians Diabetes Database from Kaggle; file: diabetes.csv.	Median imputation + StandardScaler + balanced Logistic Regression.

Accuracy	Precision	Recall	F1-score	ROC-AUC
0.734	0.603	0.704	0.650	0.813

Test-set results obtained by running the code in this report.

Contents

1	Introduction
2	Objectives
3	Dataset and Input Features
4	Methodology (Workflow)
5	Implementation: Code, Explanation and Outputs
6	Results
7	Key Findings and Observations
8	Limitation
9	Conclusion
10	Assignment Requirement Check

1. Introduction

Diabetes is a long-term health condition in which the body cannot keep blood sugar at a healthy level. In this project, a machine learning model is trained on past records to predict whether a person has diabetes (**Outcome = 1**) or does not have diabetes (**Outcome = 0**) from simple measurements such as glucose, BMI and age.

The model used is **Logistic Regression**. Despite its name, it is a *classification* method. It first combines the input features into one score, then converts that score into a probability between 0 and 1 using the sigmoid function. If the probability is 0.5 or more, the model predicts diabetes (1); otherwise it predicts no diabetes (0). Logistic Regression is popular for learning because its coefficients can be read and explained.

The data comes from the Pima Indians Diabetes Database on Kaggle (file **diabetes.csv**). The project follows the workflow given in the assignment: load the CSV, inspect the data, replace invalid zeros, split into train and test sets, apply median imputation, scale the features, train Logistic Regression, evaluate it and interpret the coefficients.

2. Objectives

- Predict the **Outcome** column: 0 = No diabetes, 1 = Diabetes.
- Use the Pima Indians Diabetes Database from Kaggle (file: diabetes.csv).
- Build a model with **median imputation + StandardScaler + balanced Logistic Regression**.
- Evaluate the model with Accuracy, Precision, Recall, F1-score, ROC-AUC and a confusion matrix.
- Interpret the model coefficients to see which inputs matter most.
- Understand the limits of the model and why it is for learning, not for medical diagnosis.

3. Dataset and Input Features

The dataset has **768 rows** and **9 columns**: eight input features and one target column (Outcome). The eight inputs named in the assignment are listed below.

Feature	Meaning
Pregnancies	Number of times pregnant
Glucose	Plasma glucose concentration
BloodPressure	Diastolic blood pressure (mm Hg)
SkinThickness	Triceps skin-fold thickness (mm)
Insulin	Serum insulin level
BMI	Body mass index
DiabetesPedigreeFunction	A score showing diabetes family history
Age	Age in years
Outcome (target)	0 = No diabetes, 1 = Diabetes

4. Methodology (Workflow)

The workflow below is the one required by the assignment. Each step is carried out in Section 5.

1. Load Kaggle CSV	2. Inspect data	3. Replace invalid zeros	4. Train-test split	5. Median imputation
6. Scaling	7. Train Logistic Regression	8. Evaluate	9. Interpret coefficients	

Important design choices (explained simply)

- **Invalid zeros:** Glucose, BloodPressure, SkinThickness, Insulin and BMI cannot truly be 0 for a living person, so zeros in these columns are treated as missing values. Pregnancies can really be 0, so it is left unchanged.
- **Split before imputing and scaling:** the median and the scaling values are learned from the training set only and then applied to the test set. This stops information from the test set leaking into training.
- **Median imputation:** missing values are filled with the median, which is not pulled around by extreme values such as the very large Insulin readings.
- **StandardScaler:** every feature is rescaled to mean 0 and standard deviation 1, so that coefficients can be compared fairly.
- **class_weight = balanced:** the data has more non-diabetic than diabetic cases, so the model gives extra weight to the smaller class and does not ignore it.

5. Implementation: Code, Explanation and Outputs

Every block below is the actual code that was executed, followed by the actual output it produced. Read from top to bottom, the code blocks form the complete working program (also supplied as an executed Jupyter notebook).

Step 1: Import libraries

What this step does: pandas and NumPy handle the data, Matplotlib and Seaborn draw charts, and scikit-learn provides the imputer, scaler, Logistic Regression model and evaluation metrics.

Code

```
# Step 1: Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             classification_report, roc_curve,
                             ConfusionMatrixDisplay)

sns.set_style("whitegrid")
print("Libraries imported successfully")
```

Output (generated by running the code)

```
Libraries imported successfully
```

Observation: All libraries loaded without errors.

Step 2: Load the Kaggle CSV

What this step does: The file diabetes.csv is read into a pandas DataFrame and its first five rows are shown.

Code

```
# Step 2: Load the Kaggle CSV (Pima Indians Diabetes Database)
df = pd.read_csv("diabetes.csv")
print("Shape (rows, columns):", df.shape)
df.head()
```

Output (generated by running the code)

```
Shape (rows, columns): (768, 9)
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

Observation: The dataset has 768 rows and 9 columns (8 inputs plus Outcome).

Step 3: Inspect the data

What this step does: We check data types, missing values, the class balance of Outcome and summary statistics.

Code

```
# Step 3: Inspect the data
print("Column data types:")
print(df.dtypes)
print()
print("Missing values (NaN) per column:")
print(df.isnull().sum())
print()
print("Class distribution of Outcome (0 = No diabetes, 1 = Diabetes):")
print(df["Outcome"].value_counts())
print()
print("Class percentage:")
print((df["Outcome"].value_counts(normalize=True) * 100).round(2))
df.describe().round(2)
```

Output (generated by running the code)

```
Column data types:
Pregnancies                   int64
Glucose                       int64
BloodPressure                 int64
SkinThickness                 int64
Insulin                       int64
BMI                         float64
DiabetesPedigreeFunction    float64
Age                           int64
Outcome                       int64
dtype: object

Missing values (NaN) per column:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64

Class distribution of Outcome (0 = No diabetes, 1 = Diabetes):
Outcome
0    500
1    268
Name: count, dtype: int64

Class percentage:
Outcome
0    65.1
1    34.9
Name: proportion, dtype: float64
       Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin     BMI  \
count       768.00   768.00         768.00         768.00   768.00  768.00
mean          3.85   120.89          69.11          20.54    79.80   31.99
std           3.37    31.97          19.36          15.95   115.24    7.88
min           0.00     0.00           0.00           0.00     0.00    0.00
25%           1.00    99.00          62.00           0.00     0.00   27.30
50%           3.00   117.00          72.00          23.00    30.50   32.00
75%           6.00   140.25          80.00          32.00   127.25   36.60
max          17.00   199.00         122.00          99.00   846.00   67.10

       DiabetesPedigreeFunction     Age  Outcome
count                    768.00  768.00   768.00
mean                       0.47   33.24     0.35
std                        0.33   11.76     0.48
min                        0.08   21.00     0.00
25%                        0.24   24.00     0.00
50%                        0.37   29.00     0.00
75%                        0.63   41.00     1.00
max                        2.42   81.00     1.00
```

Observation: All columns are numeric and there are no empty (NaN) cells yet. There are 500 non-diabetic cases (65.1%) and 268 diabetic cases (34.9%), so the classes are imbalanced. In the

statistics table, the minimum of Glucose, BloodPressure, SkinThickness, Insulin and BMI is 0, which is not possible for real measurements. These zeros are hidden missing values.

Step 3b: Count zeros in each column

What this step does: Counting zeros shows exactly which columns contain suspicious values.

Code

```
# Step 3b: Count zeros in every column (zeros can be invalid values)
print("Number of zeros in each column:")
print((df == 0).sum())
```

Output (generated by running the code)

```
Number of zeros in each column:
Pregnancies                 111
Glucose                       5
BloodPressure                35
SkinThickness               227
Insulin                     374
BMI                          11
DiabetesPedigreeFunction      0
Age                           0
Outcome                     500
dtype: int64
```

Observation: Glucose has 5 zeros, BloodPressure 35, SkinThickness 227, Insulin 374 and BMI 11. Pregnancies has 111 zeros, but 0 pregnancies is a valid value. The 500 zeros in Outcome are simply the label 'No diabetes'.

Step 3c: Visualise class balance and feature distributions

What this step does: A bar chart shows how many people are in each class, and histograms show how each feature is spread before cleaning.

Code

```
# Step 3c: Visualise the class balance and feature distributions
fig, ax = plt.subplots(figsize=(5, 3.6))
sns.countplot(x="Outcome", data=df, hue="Outcome", palette=["#4C9BE8", "#E8744C"], legend=False, ax=ax)
ax.set_xticks([0, 1])
ax.set_xticklabels(["0 = No diabetes", "1 = Diabetes"])
ax.set_title("Class Distribution of Outcome")
for p in ax.patches:
    ax.annotate(int(p.get_height()), (p.get_x() + p.get_width() / 2, p.get_height()),
                ha="center", va="bottom")
plt.tight_layout()
plt.savefig("figs/fig1_class_distribution.png", dpi=200)
plt.show()

features = df.columns.drop("Outcome")
fig, axes = plt.subplots(2, 4, figsize=(13, 6))
for ax, col in zip(axes.ravel(), features):
    ax.hist(df[col], bins=20, color="#4C9BE8", edgecolor="white")
    ax.set_title(col)
fig.suptitle("Feature Distributions (before replacing invalid zeros)", fontsize=13)
plt.tight_layout()
plt.savefig("figs/fig2_histograms_raw.png", dpi=200)
plt.show()
```

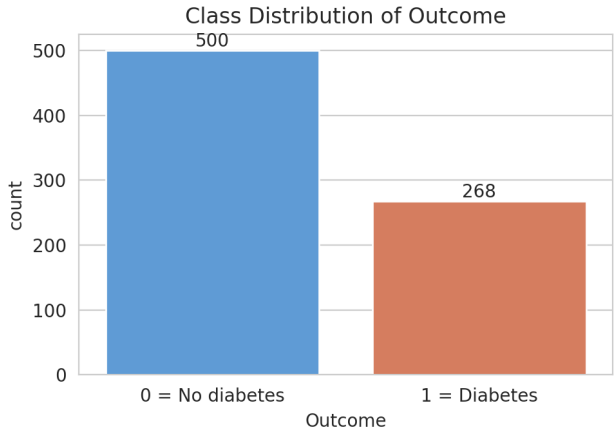


Figure 1: Class distribution of Outcome

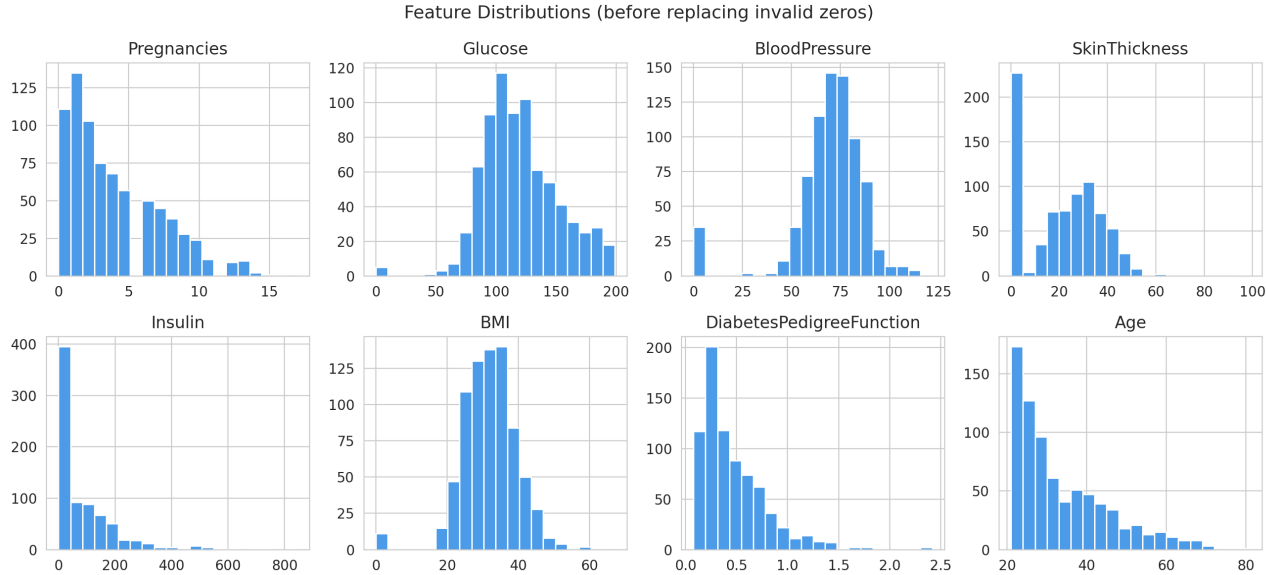


Figure 2: Feature distributions before replacing invalid zeros

Observation: Figure 1 confirms the 500 vs 268 class imbalance. In Figure 2 the tall bars at 0 for SkinThickness and Insulin (and small ones for Glucose, BloodPressure and BMI) show the invalid zeros.

Step 4: Replace invalid zeros with NaN

What this step does: Zeros in Glucose, BloodPressure, SkinThickness, Insulin and BMI are converted to NaN (missing) so they can be filled properly later.

Code

```
# Step 4: Replace invalid zeros with NaN
# A value of 0 is not possible for these body measurements, so it means "missing".
# Pregnancies can validly be 0, so it is NOT changed.
invalid_zero_cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
df[invalid_zero_cols] = df[invalid_zero_cols].replace(0, np.nan)

print("Missing values after replacing invalid zeros:")
print(df.isnull().sum())
print()
print("Percentage missing:")
print((df.isnull().mean() * 100).round(2))
```

Output (generated by running the code)

```
Missing values after replacing invalid zeros:
Pregnancies                 0
Glucose                     5
BloodPressure              35
SkinThickness             227
```

```
Insulin                     374
BMI                          11
DiabetesPedigreeFunction      0
Age                           0
Outcome                       0
dtype: int64

Percentage missing:
Pregnancies                  0.00
Glucose                      0.65
BloodPressure                4.56
SkinThickness               29.56
Insulin                     48.70
BMI                          1.43
DiabetesPedigreeFunction     0.00
Age                          0.00
Outcome                      0.00
dtype: float64
```

Observation: Insulin is missing in about 48.7% of rows and SkinThickness in about 29.6%. Glucose (0.65%), BMI (1.43%) and BloodPressure (4.56%) have fewer missing values.

Step 5: Correlation heatmap

What this step does: The heatmap shows how strongly each pair of columns moves together. Values near +1 or -1 mean a strong relationship.

Code

```
# Step 5: Correlation heatmap (uses available values only)
plt.figure(figsize=(8, 6.2))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm", square=True, linewidths=0.5)
plt.title("Correlation Heatmap of Features and Outcome")
plt.tight_layout()
plt.savefig("figs/fig3_correlation_heatmap.png", dpi=200)
plt.show()
print(df.corr()["Outcome"].drop("Outcome").sort_values(ascending=False).round(3))
```

Output (generated by running the code)

```
Glucose                   0.495
BMI                       0.314
Insulin                   0.303
SkinThickness             0.259
Age                       0.238
Pregnancies               0.222
DiabetesPedigreeFunction  0.174
BloodPressure             0.171
Name: Outcome, dtype: float64
```

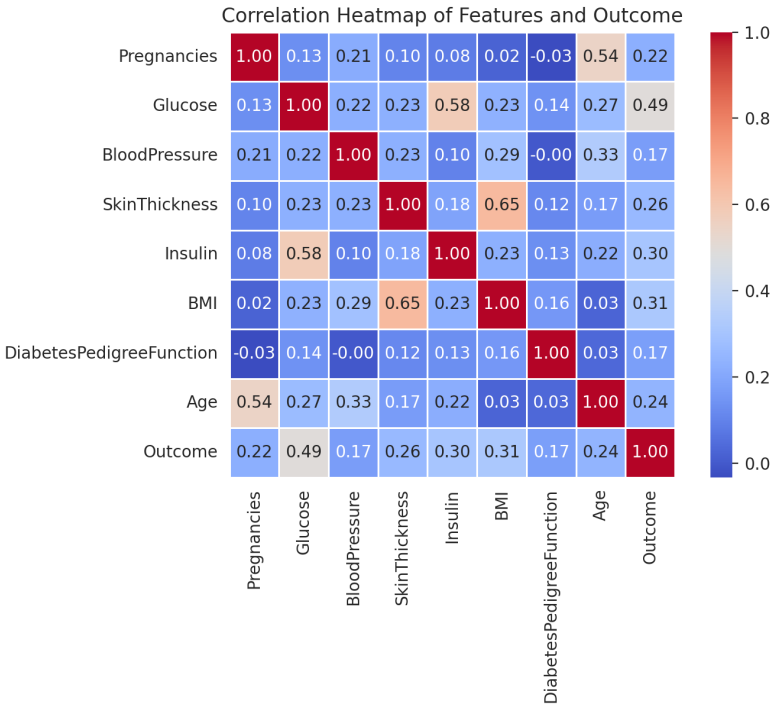


Figure 3: Correlation heatmap

Observation: Glucose has the strongest correlation with Outcome (0.495), followed by BMI (0.314) and Insulin (0.303). Some inputs are also related to each other, for example SkinThickness and BMI (0.65), Glucose and Insulin (0.58), and Age and Pregnancies (0.54).

Step 6: Train-test split

What this step does: The data is split 80% for training and 20% for testing. stratify=y keeps the same class ratio in both parts and random_state=42 makes the split repeatable (the 'fixed test split').

Code

```
# Step 6: Train-test split (80% train, 20% test)
# stratify keeps the same diabetes/no-diabetes ratio in both sets
# random_state=42 fixes the split so results can be repeated
X = df.drop(columns="Outcome")
y = df["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print("X_train shape:", X_train.shape)
print("X_test shape :", X_test.shape)
print()
print("Training class counts:")
print(y_train.value_counts())
print("Test class counts:")
print(y_test.value_counts())
```

Output (generated by running the code)

```
X_train shape: (614, 8)
X_test shape : (154, 8)

Training class counts:
Outcome
0    400
1    214
Name: count, dtype: int64
Test class counts:
Outcome
0    100
1     54
Name: count, dtype: int64
```

Observation: There are 614 training rows (400 non-diabetic, 214 diabetic) and 154 test rows (100 non-diabetic, 54 diabetic). Both parts keep roughly the same class ratio.

Step 7: Median imputation

What this step does: SimpleImputer(strategy='median') learns the median of every column from the training set and fills the missing values in both the training and test sets.

Code

```
# Step 7: Median imputation (fit on TRAIN data only, to avoid data leakage)
imputer = SimpleImputer(strategy="median")
X_train_imp = imputer.fit_transform(X_train)
X_test_imp = imputer.transform(X_test)

print("Median values learned from the training set:")
print(pd.Series(imputer.statistics_, index=X.columns).round(3))
print()
print("Missing values left in training data:", int(np.isnan(X_train_imp).sum()))
print("Missing values left in test data    :", int(np.isnan(X_test_imp).sum()))
```

Output (generated by running the code)

```
Median values learned from the training set:
Pregnancies                 3.000
Glucose                   117.000
BloodPressure              72.000
SkinThickness              29.000
Insulin                   125.000
BMI                        32.400
DiabetesPedigreeFunction    0.382
Age                        29.000
dtype: float64

Missing values left in training data: 0
Missing values left in test data    : 0
```

Observation: The learned medians are listed above (for example Glucose 117, Insulin 125). No missing values remain in either set.

Step 8: Scaling with StandardScaler

What this step does: Each feature is rescaled so that its mean is 0 and standard deviation is 1. Again, the scaler learns from the training set only.

Code

```
# Step 8: Scaling with StandardScaler (mean = 0, std = 1; fit on TRAIN only)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_imp)
X_test_scaled = scaler.transform(X_test_imp)

print("Mean of scaled training features (should be ~0):")
print(pd.Series(X_train_scaled.mean(axis=0), index=X.columns).round(4))
print()
print("Std of scaled training features (should be ~1):")
print(pd.Series(X_train_scaled.std(axis=0), index=X.columns).round(4))
```

Output (generated by running the code)

```
Mean of scaled training features (should be ~0):
Pregnancies               -0.0
Glucose                   -0.0
BloodPressure              0.0
SkinThickness             -0.0
Insulin                   -0.0
BMI                        0.0
DiabetesPedigreeFunction  -0.0
Age                       -0.0
dtype: float64

Std of scaled training features (should be ~1):
Pregnancies                1.0
Glucose                    1.0
BloodPressure              1.0
SkinThickness              1.0
Insulin                    1.0
BMI                        1.0
```

```
DiabetesPedigreeFunction    1.0
Age                         1.0
dtype: float64
```

Observation: The scaled training features have mean about 0 and standard deviation 1, as expected.

Step 9: Train Logistic Regression

What this step does: The model is trained on the scaled training data. class_weight='balanced' gives more importance to the diabetic class, and max_iter=1000 allows enough steps for the solver to finish.

Code

```
# Step 9: Train Logistic Regression (class_weight="balanced" handles the class imbalance)
model = LogisticRegression(class_weight="balanced", max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)

print("Model trained successfully")
print("Intercept:", model.intercept_.round(4))
```

Output (generated by running the code)

```
Model trained successfully
Intercept: [-0.257]
```

Observation: The model trained successfully and the learned intercept is -0.257.

Step 10: Predictions on the test set

What this step does: The trained model predicts the class and the probability of diabetes for the 154 unseen test rows.

Code

```
# Step 10: Predictions on the test set
y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[:, 1]

print("First 10 actual values   :", y_test.values[:10])
print("First 10 predicted values:", y_pred[:10])
print("First 10 probabilities   :", y_prob[:10].round(3))
```

Output (generated by running the code)

```
First 10 actual values   : [0 0 0 1 0 0 1 1 0 0]
First 10 predicted values: [1 0 0 0 0 0 1 1 0 1]
First 10 probabilities   : [0.733 0.212 0.439 0.383 0.055 0.281 0.656 0.962 0.132 0.907]
```

Observation: For the first 10 test rows, 7 predictions match the actual values. The probabilities show how confident the model is.

Step 11: Evaluation metrics

What this step does: Accuracy, Precision, Recall, F1-score and ROC-AUC are calculated on the test set, followed by a classification report.

Code

```
# Step 11: Evaluation metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_prob)

metrics = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-score", "ROC-AUC"],
    "Result": [round(acc, 3), round(prec, 3), round(rec, 3), round(f1, 3), round(auc, 3)]})
print(metrics.to_string(index=False))
print()
print("Classification report:")
print(classification_report(y_test, y_pred, target_names=["No diabetes (0)", "Diabetes (1)"]))
```

Output (generated by running the code)

```
   Metric  Result
 Accuracy   0.734
Precision   0.603
   Recall   0.704
 F1-score   0.650
  ROC-AUC   0.813

Classification report:
                precision    recall  f1-score   support

No diabetes (0)       0.82      0.75      0.79       100
   Diabetes (1)       0.60      0.70      0.65        54

       accuracy                           0.73       154
      macro avg       0.71      0.73      0.72       154
   weighted avg       0.75      0.73      0.74       154
```

Observation: Accuracy = 0.734, Precision = 0.603, Recall = 0.704, F1-score = 0.650 and ROC-AUC = 0.813.

Step 12: Confusion matrix

What this step does: The confusion matrix counts correct and wrong predictions for each class.

Code

```
# Step 12: Confusion matrix
cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()
print("Confusion matrix:")
print(cm)
print(f"TN = {tn}, FP = {fp}, FN = {fn}, TP = {tp}")

fig, ax = plt.subplots(figsize=(5, 4.2))
ConfusionMatrixDisplay(cm, display_labels=["No diabetes", "Diabetes"]).plot(
    ax=ax, cmap="Blues", colorbar=False, values_format="d")
ax.grid(False)
ax.set_title("Confusion Matrix (Test Set)")
plt.tight_layout()
plt.savefig("figs/fig4_confusion_matrix.png", dpi=200)
plt.show()
```

Output (generated by running the code)

```
Confusion matrix:
[[75 25]
 [16 38]]
TN = 75, FP = 25, FN = 16, TP = 38
```

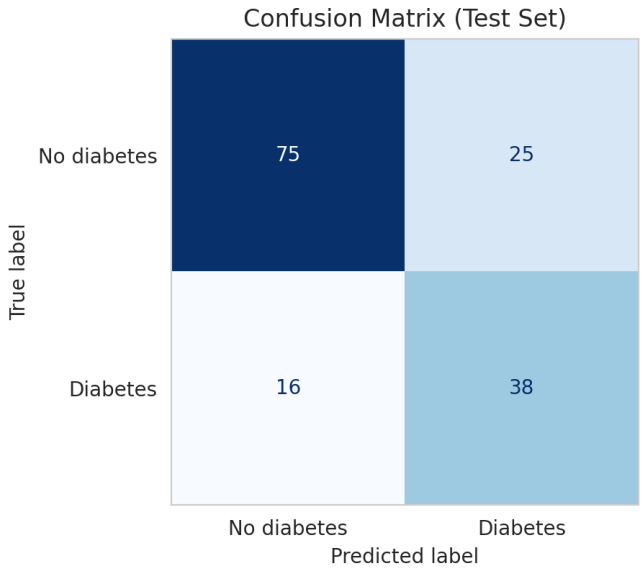


Figure 4: Confusion matrix on the test set

Observation: TN = 75, FP = 25, FN = 16, TP = 38. Of the 54 people who really had diabetes, the model found 38 and missed 16.

Step 13: ROC curve

What this step does: The ROC curve plots the true positive rate against the false positive rate at all thresholds. The closer the curve is to the top-left corner, the better the model separates the two classes.

Code

```
# Step 13: ROC curve
fpr, tpr, _ = roc_curve(y_test, y_prob)
plt.figure(figsize=(5.5, 4.5))
plt.plot(fpr, tpr, color="#E8744C", lw=2, label=f"Logistic Regression (AUC = {auc:.3f})")
plt.plot([0, 1], [0, 1], "k--", label="Random guess (AUC = 0.5)")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate (Recall)")
plt.title("ROC Curve")
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig("figs/fig5_roc_curve.png", dpi=200)
plt.show()
```

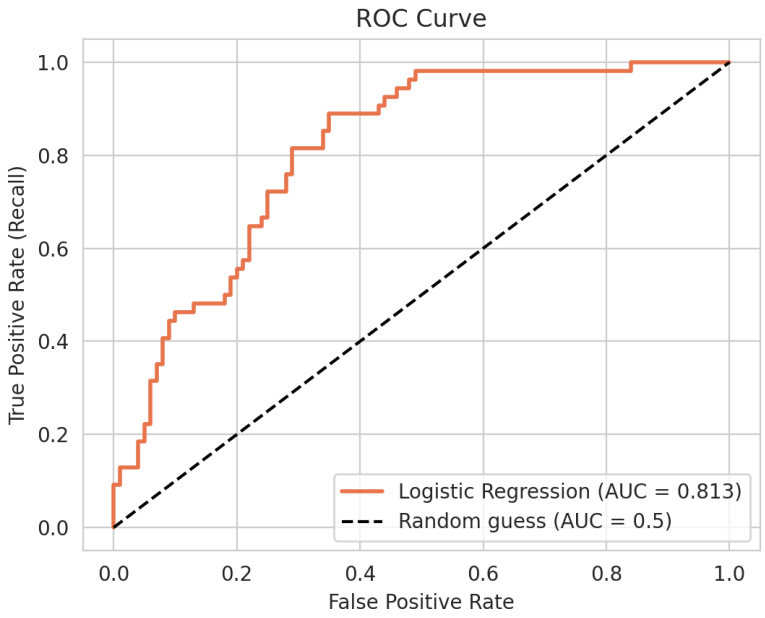


Figure 5: ROC curve

Observation: The curve stays well above the random-guess diagonal, with an AUC of 0.813.

Step 14: Interpret the coefficients

What this step does: Each coefficient shows how a feature changes the log-odds of diabetes. Because the features are standardised, the coefficients are directly comparable. The odds ratio (exp of the coefficient) shows how the odds change for a one-standard-deviation increase in that feature.

Code

```
# Step 14: Interpret the coefficients
coef_df = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_[0],
    "Odds Ratio (exp(coef))": np.exp(model.coef_[0])
}).sort_values("Coefficient", ascending=False).reset_index(drop=True)
print(coef_df.round(4).to_string(index=False))

plot_df = coef_df.sort_values("Coefficient")
colors = ["#E8744C" if c > 0 else "#4C9BE8" for c in plot_df["Coefficient"]]
plt.figure(figsize=(7, 4.5))
plt.barh(plot_df["Feature"], plot_df["Coefficient"], color=colors)
plt.axvline(0, color="black", lw=0.8)
plt.xlabel("Coefficient (on standardised features)")
plt.title("Logistic Regression Coefficients")
plt.tight_layout()
plt.savefig("figs/fig6_coefficients.png", dpi=200)
```

```
plt.show()
```

Output (generated by running the code)

```
                  Feature  Coefficient  Odds Ratio (exp(coef))
                  Glucose       1.1835                  3.2657
                      BMI       0.7096                  2.0332
              Pregnancies       0.3729                  1.4519
DiabetesPedigreeFunction       0.2877                  1.3334
                      Age       0.1864                  1.2049
            SkinThickness       0.0140                  1.0141
            BloodPressure      -0.0146                  0.9855
                  Insulin      -0.0446                  0.9564
```

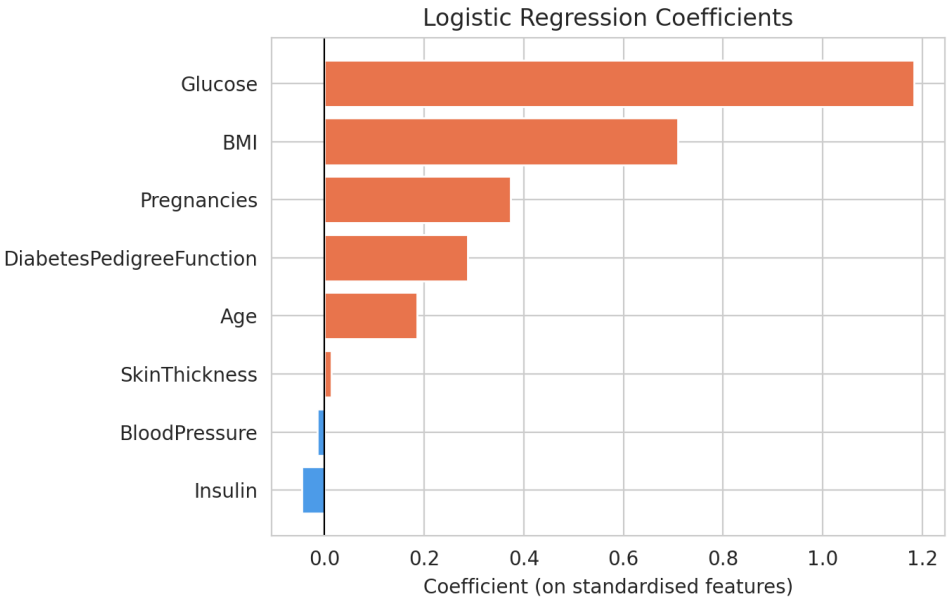


Figure 6: Logistic Regression coefficients

Observation: Glucose (1.183) and BMI (0.710) have the largest positive coefficients. Pregnancies, DiabetesPedigreeFunction and Age have smaller positive effects. SkinThickness, BloodPressure and Insulin are close to zero.

Step 15: Save the final numbers

What this step does: The final results are written to a small JSON file so they can be reused in this report.

Code

```
# Step 15: Save final numbers (used to build the report)
import json
results = {
    "rows": int(df.shape[0]), "cols": int(df.shape[1]),
    "train": int(len(y_train)), "test": int(len(y_test)),
    "accuracy": float(acc), "precision": float(prec), "recall": float(rec),
    "f1": float(f1), "auc": float(auc),
    "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp),
    "coef": coef_df.to_dict(orient="records")}
with open("results.json", "w") as f:
    json.dump(results, f, indent=2)
print("results.json saved")
```

Output (generated by running the code)

```
results.json saved
```

Observation: The results file was saved.

6. Results

6.1 Evaluation metrics (test set, 154 rows)

Metric	Result	Meaning
Accuracy	0.734	Overall correct predictions
Precision	0.603	Reliability of positive predictions
Recall	0.704	Actual positive cases detected
F1-score	0.650	Balance of precision and recall
ROC-AUC	0.813	Overall class-separation ability

6.2 Confusion matrix

	Predicted 0	Predicted 1
Actual 0	TN = 75	FP = 25
Actual 1	FN = 16	TP = 38

How to read these numbers: 113 of 154 test predictions are correct (75+38), which gives the accuracy of 0.734. The model predicted diabetes 63 times and was right 38 times (precision 38/63 = 0.603). There are 54 real diabetic cases in the test set and the model found 38 of them (recall 38/54 = 0.704). Among the 100 non-diabetic people, 75 were correctly cleared (75/100 = 0.75).

6.3 Coefficients

Feature	Coefficient	Odds ratio
Glucose	1.1835	3.2657
BMI	0.7096	2.0332
Pregnancies	0.3729	1.4519
DiabetesPedigreeFunction	0.2877	1.3334
Age	0.1864	1.2049
SkinThickness	0.0140	1.0141
BloodPressure	-0.0146	0.9855
Insulin	-0.0446	0.9564

A positive coefficient raises the predicted chance of diabetes; a negative coefficient lowers it. The odds ratios are for a one-standard-deviation increase in the feature, with the other features held fixed.

7. Key Findings and Observations

- The model reaches a ROC-AUC of about 0.813 and detects about 70.4% of the positive cases in the fixed test split (38 of 54).
- Glucose and BMI have the strongest positive model coefficients (1.183 and 0.710). Higher glucose and higher BMI both push the prediction towards diabetes. Glucose is also the feature most correlated with Outcome (0.495).
- Recall is especially important in a screening-style task, because a false negative is a missed positive case. Using balanced class weights helped here: the model found 38 of 54 diabetic cases,

but it also raised 25 false alarms (FP = 25), which is the price of catching more real cases.

- Precision is 0.603, so about 4 in 10 positive predictions are false alarms. In a screening setting this is usually more acceptable than missing real cases.
- A large part of the data needed repair: Insulin had about 48.7% missing values and SkinThickness about 29.6% once the invalid zeros were marked. These values were filled with training-set medians, so those two features carry less reliable information.
- The coefficients of SkinThickness, BloodPressure and Insulin are close to zero. This may be because they carry little extra signal, because many of their values were imputed, or because they overlap with other features (for example Insulin with Glucose, correlation 0.58). The results alone do not prove which reason is correct.

8. Limitation

This is a small historical dataset (768 rows) that represents a specific population, and the results come from one fixed train-test split with only 154 test rows, so the numbers could change with a different split. The model is suitable for **learning classification concepts, not for medical diagnosis**.

9. Conclusion

In this project I built a Logistic Regression model to predict diabetes from the Pima Indians Diabetes data. I first found that some columns contained zeros that are not possible for real measurements, treated them as missing values, and filled them with the median after the train-test split. I then scaled the features and trained a balanced Logistic Regression model. On the test set it reached an accuracy of 0.734, precision of 0.603, recall of 0.704, F1-score of 0.650 and ROC-AUC of 0.813. The model found 38 of the 54 diabetic cases, missed 16, and gave 25 false alarms. The coefficients show that Glucose and BMI are the most important positive factors. Because recall matters in a screening-style task, the balanced model is a sensible choice, but the dataset is small and specific, so this model is only for learning and should not be used for real medical diagnosis.

10. Assignment Requirement Check

Requirement from the assignment	Where it is covered
Objective: predict Outcome (0 = No diabetes, 1 = Diabetes)	Section 2; title page
Dataset: Pima Indians Diabetes Database, diabetes.csv	Section 3; Step 2
Model: median imputation + StandardScaler + balanced Logistic Regression	Steps 7, 8, 9
Inputs: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age	Section 3
Workflow: load, inspect, replace invalid zeros, split, impute, scale, train, evaluate, interpret	Section 4; Steps 2 to 14
Evaluation metrics table: Accuracy, Precision, Recall, F1-score, ROC-AUC	Step 11; Section 6.1
Confusion matrix (TN, FP, FN, TP)	Step 12; Section 6.2
Interpret coefficients (Glucose and BMI strongest positive)	Step 14; Section 6.3
Key findings, including the importance of recall	Section 7
Limitation: small dataset, learning only, not for diagnosis	Section 8
Student name and date added	Title page
Metric outputs and graphs visible	Sections 5 and 6 (Figures 1 to 6)
Final conclusion in own words	Section 9 (please re-read it and adjust the wording to your own style)

All numbers in this report were produced by executing the code on the downloaded diabetes.csv. They match the reference values given in the assignment sheet (accuracy 0.734, precision 0.603, recall 0.704, F1 0.650, ROC-AUC 0.813, TN 75, FP 25, FN 16, TP 38).